MODUL PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBYEK

MODUL 7
Abstract Class, Abstract Method, dan Interface



Disusun oleh:

Nirwana Haidar Hari, S.Pd., M.Kom.

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
PROGRAM STUDI DI LUAR KAMPUS UTAMA SUMENEP
PENS PSDKU SUMENEP

2026

MODUL PRAKTIKUM P7
Abstract Class, Abstract Method, dan Interface
Project Based Learning + Agile

1. Tujuan Praktikum

Setelah menyelesaikan praktikum, mahasiswa mampu:

a. Membedakan class biasa, abstract class, dan interface.
b. Membuat abstract class dan abstract method.
c. Mengimplementasikan abstract method pada subclass.
d. Membuat dan menggunakan interface dengan keyword `implements`.
e. Menggunakan polymorphism melalui abstract class dan interface.
f. Melakukan refactoring proyek P6 berdasarkan prinsip abstraction.
g. Melaksanakan Sprint Review dan Sprint Retrospective sebagai bagian dari pengembangan proyek.

2. Review P6

Pada P6, mahasiswa telah menerapkan inheritance, method overriding, overloading, upcasting, dan dynamic binding.

Contoh:

```
class Kendaraan {
    public void bergerak() {
        System.out.println("Kendaraan bergerak");
    }
}

class Mobil extends Kendaraan {
    @Override
    public void bergerak() {
        System.out.println("Mobil berjalan di jalan");
    }
}

class Kapal extends Kendaraan {
    @Override
    public void bergerak() {
        System.out.println("Kapal bergerak di air");
    }
}
```

Pada P7, mahasiswa perlu mempertimbangkan apakah `Kendaraan` memang harus dapat dibuat menjadi object secara langsung, atau cukup digunakan sebagai konsep umum bagi berbagai jenis kendaraan.

Jika superclass hanya berfungsi sebagai kerangka umum, mahasiswa dapat mempertimbangkan penggunaan abstract class.

3. Konsep Singkat

A. Abstract Class

Abstract class merupakan class yang tidak dapat dibuat menjadi object secara langsung menggunakan `new`.

Contoh:

```
abstract class Kendaraan {

    private String nama;

    public Kendaraan(String nama) {
        this.nama = nama;
    }

    public String getNama() {
        return nama;
    }

    public abstract void bergerak();
}
```

Abstract class dapat memiliki attribute, constructor, method biasa, dan abstract method.

Kode berikut tidak diperbolehkan:

```
Kendaraan kendaraan = new Kendaraan("Kendaraan");
```

Namun, abstract class dapat digunakan sebagai tipe reference:

```
Kendaraan kendaraan = new Mobil("Mobil");
```

B. Abstract Method

Abstract method merupakan deklarasi method tanpa implementasi langsung.

public abstract void bergerak();

Subclass konkret wajib mengimplementasikan seluruh abstract method yang diwarisinya.

```
class Mobil extends Kendaraan {

    public Mobil(String nama) {
        super(nama);
    }
```

```
    @Override
    public void bergerak() {
        System.out.println(
            getNama() + " berjalan di jalan"
        );
    }
}
```

C. Interface

Interface mendefinisikan kontrak perilaku yang dapat diterapkan oleh berbagai class.

Contoh:

```
interface DapatDilacak {

    String getLokasi();
}
```

Implementasi:

```
class Mobil extends Kendaraan
        implements DapatDilacak {

    public Mobil(String nama) {
        super(nama);
    }

    @Override
    public void bergerak() {
        System.out.println(
            getNama() + " berjalan di jalan"
        );
    }

    @Override
    public String getLokasi() {
        return "Area Parkir";
    }
}
```

Java memungkinkan satu class mengimplementasikan lebih dari satu interface.

D. Perbedaan Abstract Class dan Interface

Aspek	Abstract Class	Interface
Fungsi utama	Mewakili konsep umum dan berbagi implementasi	Menentukan kontrak perilaku

Keyword	`abstract` dan `extends`	`interface` dan `implements`
Attribute instance	Dapat memiliki	Tidak dapat memiliki
Constructor	Dapat memiliki	Tidak memiliki
Method dengan implementasi	Dapat memiliki	Dapat melalui `default`, `static`, atau `private` method
Hubungan	Satu superclass langsung	Dapat mengimplementasikan beberapa interface

Catatan: Interface bukan sekadar abstract class yang seluruh method-nya abstrak. Java modern juga mendukung default method dan static method pada interface.

4. Contoh Implementasi Lengkap

Buat empat file Java berikut.

Kendaraan.java

```
public abstract class Kendaraan {

    private String nama;

    public Kendaraan(String nama) {
        this.nama = nama;
    }

    public String getNama() {
        return nama;
    }

    public abstract void bergerak();
}
```

DapatDilacak.java

```
public interface DapatDilacak {
    String getLokasi();
}
```

Mobil.java

```
public class Mobil extends Kendaraan
        implements DapatDilacak {

    public Mobil(String nama) {
        super(nama);
    }
```

```
    @Override
    public void bergerak() {
        System.out.println(
            getNama() + " berjalan di jalan"
        );
    }

    @Override
    public String getLokasi() {
        return "Area Parkir";
    }
}
```

Main.java

```
public class Main {

    public static void main(String[] args) {

        Kendaraan kendaraan =
            new Mobil("Mobil Operasional");

        kendaraan.bergerak();

        DapatDilacak pelacak =
            new Mobil("Mobil Operasional");

        System.out.println(
            pelacak.getLokasi()
        );
    }
}
```

Hasil yang diharapkan:

```
Mobil Operasional berjalan di jalan
Area Parkir
```

Perhatikan bahwa object `Mobil` dapat digunakan melalui reference `Kendaraan` maupun `DapatDilacak`. Keduanya menunjukkan bentuk polymorphism.

5. Agile Sprint – P7

Sprint Goal

Mengembangkan desain proyek P6 dengan menerapkan abstract class dan interface yang sesuai sehingga struktur class lebih jelas, perilaku object memiliki kontrak yang konsisten, dan program tetap dapat dijalankan.

Sprint Backlog

Sprint Backlog P7
0/7 selesai
SB-01 · Audit hierarchy dan behavior proyek P6
To Do
SB-02 · Menentukan kandidat abstract class
To Do
SB-03 · Menentukan abstract method
To Do
SB-04 · Menentukan kandidat interface
To Do
SB-05 · Memperbarui class diagram
To Do
SB-06 · Mengimplementasikan abstract class dan interface
To Do
SB-07 · Menguji polymorphism dan program
To Do
Salin Sprint Backlog

6. Tugas Praktikum

Tugas P7 – Refactoring Proyek dengan Abstract Class dan Interface

Gunakan proyek hasil P6. Mahasiswa tidak membuat proyek baru, melainkan memperbaiki struktur class dan menambahkan perilaku sesuai kebutuhan sistem.

Bagian A – Audit Desain P6

Analisis superclass dan subclass yang telah dibuat. Tentukan class yang seharusnya tidak dapat diinstansiasi secara langsung dan perilaku yang dapat didefinisikan sebagai kontrak melalui interface.

Buat tabel berikut.

Class / Behavior	Kondisi P6	Rencana P7	Alasan
Kendaraan	Class biasa	Abstract class	Hanya mewakili konsep umum
bergerak()	Method biasa	Abstract method	Setiap subclass memiliki cara bergerak berbeda
DapatDilacak	Belum ada	Interface	Kontrak pelacakan dapat digunakan berbagai jenis object

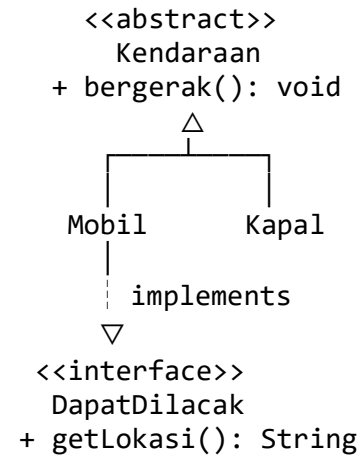
Ketentuan: Setiap keputusan perubahan desain harus didukung alasan. Jangan mengubah seluruh superclass menjadi abstract class tanpa mempertimbangkan kebutuhan proyek.

Bagian B – Update Class Diagram

Perbarui class diagram dengan menampilkan:

- minimal 1 abstract class yang relevan;
- minimal 1 abstract method;
- minimal 2 subclass konkret;
- minimal 1 interface;
- relasi generalization dan realization;
- relasi antarobject dari P4 yang masih digunakan.

Contoh notasi UML sederhana:



Pada diagram UML final, gunakan garis generalization dan realization sesuai notasi UML yang benar.

Bagian C – Implementasi Abstract Class

Buat atau refactor minimal 1 superclass menjadi abstract class. Abstract class harus memiliki constructor, attribute atau method umum yang relevan, dan minimal 1 abstract method.

Contoh:

```
public abstract class Kendaraan {

    private String nama;

    public Kendaraan(String nama) {
        this.nama = nama;
    }

    public String getNama() {
        return nama;
    }
```

```
    public abstract void bergerak();
}
```

Bagian D – Implementasi Subclass

Buat minimal 2 subclass konkret yang mengimplementasikan abstract method sesuai karakteristik masing-masing.

Contoh:

```
public class Kapal extends Kendaraan {

    public Kapal(String nama) {
        super(nama);
    }

    @Override
    public void bergerak() {
        System.out.println(
            getNama() + " bergerak di air"
        );
    }
}
```

Subclass harus dapat dibuat menjadi object dan dijalankan tanpa kesalahan kompilasi.

Bagian E – Implementasi Interface

Buat minimal 1 interface dengan method yang relevan terhadap proyek.

```
public interface DapatDilacak {
   String getLokasi();
}
```

Implementasikan interface pada minimal 2 class yang sesuai. Kedua class tersebut tidak harus memiliki superclass yang sama, tetapi harus memenuhi kontrak interface yang digunakan.

Bagian F – Pengujian Polymorphism

Buat minimal 4 skenario pengujian.

No.	Skenario	Hasil yang Diharapkan
1	Membuat object subclass	Object berhasil dibuat
2	Memanggil abstract method melalui reference superclass	Method subclass yang sesuai dijalankan
3	Memanggil method melalui reference interface	Implementasi interface yang sesuai dijalankan

4	Menggunakan beberapa object dalam array/list bertipe superclass atau interface	Setiap object menjalankan perilakunya sendiri

Contoh:

```
Kendaraan[] daftar = {
    new Mobil("Mobil A"),
    new Kapal("Kapal B")
};

for (Kendaraan kendaraan : daftar) {
    kendaraan.bergerak();
}
```

Mahasiswa harus dapat menjelaskan mengapa method yang sama menghasilkan perilaku berbeda.

7. Luaran Tugas

Kumpulkan satu folder berikut.

```
P7_NRP_Nama/
│
├── README.pdf
│
└── src/
    ├── Main.java
    ├── AbstractClass.java
    ├── Interface.java
    ├── Subclass1.java
    ├── Subclass2.java
    └── ClassLain.java
```

Nama file Java disesuaikan dengan nama class dan interface yang digunakan pada proyek masing-masing.

`README.pdf` maksimal 3 halaman.

Halaman	Konten
1	Nama proyek, Sprint Goal, Sprint Backlog, dan tabel audit desain P6

2	Class diagram terbaru, abstract class, abstract method, interface, dan alasan perubahan desain
3	Screenshot hasil running, tabel pengujian, Sprint Review, dan Sprint Retrospective

8. Sprint Review

Mahasiswa mendemonstrasikan hasil implementasi dan menunjukkan bahwa perubahan desain tidak merusak fitur yang telah dibuat pada P1–P6.

Checklist Sprint Review

- [ ] Abstract class berhasil dibuat
- [ ] Abstract method diimplementasikan subclass
- [ ] Minimal 2 subclass dapat digunakan
- [ ] Interface berhasil diimplementasikan
- [ ] Class diagram sesuai dengan kode
- [ ] Pengujian polymorphism berhasil
- [ ] Fitur proyek sebelumnya tetap berjalan
- [ ] 0/7 item terverifikasi

9. Sprint Retrospective

Jawab masing-masing secara singkat.

What Went Well?

- Implementasi abstraction atau interface apa yang berhasil?

What Went Wrong?

- Apa kendala dalam menentukan abstract class dan interface?

Improvement

- Apa yang akan diperbaiki pada sprint berikutnya?
- Salin Sprint Retrospective

10. Definition of Done

Tugas P7 dinyatakan selesai apabila seluruh persyaratan berikut terpenuhi.

Checklist penyelesaian tugas

0/15
Menggunakan proyek hasil P6
Audit desain dan alasan penggunaan abstraksi tersedia
Minimal 1 abstract class yang relevan dibuat
Minimal 1 abstract method dibuat
Minimal 2 subclass konkret mengimplementasikan abstract method
Minimal 1 interface dibuat
Minimal 2 class mengimplementasikan interface yang relevan
Polymorphism melalui superclass dan interface dapat dibuktikan
Class diagram diperbarui dan konsisten dengan kode
Program berhasil dikompilasi dan dijalankan
Minimal 4 skenario pengujian tersedia
Fitur proyek sebelumnya tetap berjalan
Sprint Backlog diperbarui
Sprint Review tersedia
Sprint Retrospective tersedia

11. Penilaian

Komponen	Bobot
Audit desain dan ketepatan pemilihan abstraction	15%
Implementasi abstract class dan abstract method	25%
Implementasi interface	20%
Polymorphism dan pengujian program	20%

Class diagram dan konsistensi desain	10%
Agile Sprint dan dokumentasi	10%
Total	100%

Penilaian menekankan ketepatan keputusan desain dan keberhasilan implementasi, bukan sekadar jumlah abstract class atau interface yang dibuat.

12. Target Pertemuan Berikutnya

P8 — UTS Praktikum

Evaluasi integrasi kompetensi P1–P7

Mahasiswa menyelesaikan studi kasus OOP yang mengintegrasikan class, object, constructor, encapsulation, relasi antarobject, inheritance, polymorphism, abstract class, dan interface.

Penilaian dilakukan melalui pemeriksaan kode, demonstrasi program, dan klarifikasi individual.

Hasil P7 menjadi baseline proyek untuk melanjutkan pengembangan pada P9, yaitu implementasi dan penyempurnaan UML Class Diagram ke dalam struktur program Java.